Machine Vision for Deep Fake Detection

Patrick Donnelly and Burke Havranek

GitHub Repository: https://github.com/EECE-5644-Summer-B/FinalProject

Abstract

In recent years, increasing concern has been placed on the use of AI generated photographs, both of real people and of manufactured individuals. Such "deep fakes" have been used to propagate false information, blackmail and scam individuals, and damage public discourse. The purpose of this project is to train a model capable of discerning between real images and deep fakes given an image of the individual's face. The utility of such a system would be to verify the veracity of photographs for every manner of purpose, from news outlets, to social media, in an attempt to resist the propagation of deep fake images. Using the below data set, *130k Real vs Fake Face* by Shreyansh Patel, we train an AI model capable of discerning between the included real and fake face images using machine vision. To do so, we sanitize the data set, which features a mixture of photograph resolutions and is unbalanced in both its number and type of fake image generation methods. Given the sanitized data set, we then train an EfficientNet-B0 classification model that, with perfect accuracy, determines whether the image in question is genuine or faked. Future labels to train for may also include sex, emotion, age, etc., though the aforementioned data set lacks labels to this end.

Data Set

The aforementioned data set is publicly available from Kaggle, and is provided under the MIT License. The data set is divided into two groups, real face images and artificially-generated face images. The real images are taken from the Flickr-Faces-HQ Dataset (FFHQ) data set, sourced from various publicly-available Flickr images and provided under several public licenses. The fake images were generated by the author using Flux1.pro, Flux1.dev, and Stable Diffusion XL (SDXL). The data is as follows:

- Real Human Faces: 70,000 256x256 JPEG images (verified by us)
- Fake Human Faces: 63,569 1024x1024 JPEG images (verified by us, differs slightly from author's claim of 64,000 total images)
    - FLUX1 PRO: 3209 Images
    - FLUX1 DEV: 7273 Images
    - SDXL: 53087 Images

Beyond the images themselves, which technically comprise features, the data set simply categorizes images as “real” or “fake,” with the model used to generate the image (i.e., FLUX1 PRO, FLUX1 DEV, or SDXL) noted in the latter case. Images do not appear to tag any other features (e.g., age, race, context), and a manual review of the images supports this.

Data Preprocessing

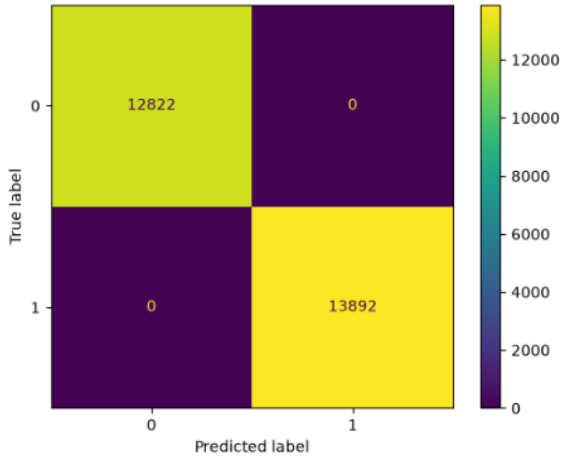
For the photo data sets, there were a few issues that needed to be addressed. The data was pre-cleaned, already being cropped and validated. In addition, outliers are difficult to quantify for image data of this variety, and are nevertheless incredibly important for proper testing, thus we did not remove any data. The fact of them being real or fake needed to be encoded, but the bulk of the preprocessing was dealing with the photos themselves. All photos were downsampled to 224x224 resolution using the Lanczos algorithm, then split into standard RGB color channels. These color values may help due to AI photos tending to exhibit higher contrast and more vibrant colors, which may help with identification over standard monochromatic analysis. For normalization, the ImageNet standard was used, as we want the final model to be generally applicable to the internet at large. The resulting normalized three-channel data was finally saved at PyTorch tensors with an accompanying table of labels for training and validation.

Machine Learning Model Implementation

For our binary classification model, we chose to begin with the EfficientNet-B0 convolutional neural network architecture, which we then trained using standard backpropagation with cross-entropy as the loss function and Adam as the hyperparameter optimizer. The EfficientNet model was chosen due to its superior performance with fewer parameters and FLOPs on machine vision problems, for which it was specifically optimized, over other such well-established model architectures; due to the scale and complexity of this categorization problem, we felt as though construction of a model from first principles would be far too ambitious, time-consuming, and most likely inferior to such models as EfficientNet to entertain. Cross-entropy and Adam were similarly chosen for being industry standard in machine vision classification and generally superior to anything the authors could construct from scratch in the time allowed. All three of the aforementioned algorithms were implemented using their PyTorch built-in implementations with default settings, for the same reasons as previous.

Model Performance Evaluation

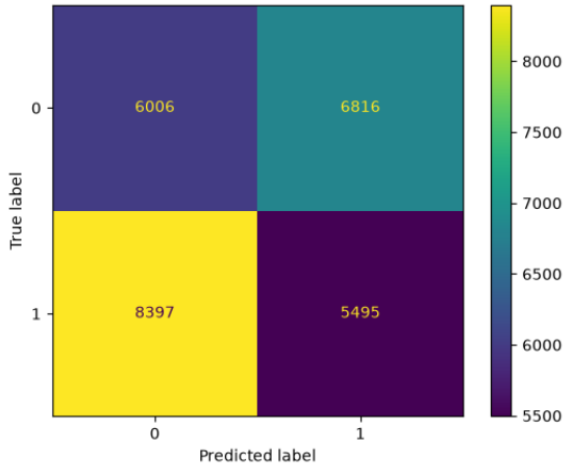
Our final model required 45 epochs of training against our data set, which we split into 80% training images and 20% validation images, as is standard. The final training and validation loss, as calculated using median cross-entropy across each batch, was exactly 0 in both cases, indicating perfect performance. In particular, we note the following confusion matrix and metrics, which support the above conclusion:




```
Final Model:
Accuracy: 1.000
Precision: 1.000
Recall: 1.000
F1 Score: 1.000
```

Hyperparameter Tuning

As described previously, due to the scale and complexity of this model, hyperparameter tuning was undertaken using the Adam algorithm while training. Manual tuning, while considered, was forwent due to the training time required of the model, taking on average approximately 9 hours per iteration. For these reasons, we do not have several models with differing hyperparameters to compare. Instead, we compare against the pre-trained default EfficientNet-B0 model, which produces the following metrics:

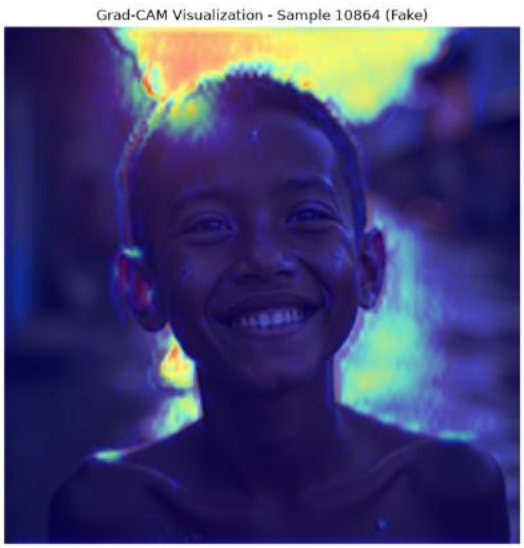



```
Control Model:
Accuracy: 0.431
Precision: 0.446
Recall: 0.396
F1 Score: 0.419
```

From the above, we find the default model to be greatly inferior to our trained model, with the control performing significantly worse than random chance, capturing approximately 40% of the data across all metrics. By comparison with our apparently perfect model, at least as far as our validation set can determine, the control model is intractable.

Feature Importance

In the machine learning sense, the only "feature" on which our model operates in the preprocessed image data. To that end, our model operates as a black box. Performing feature extraction using the Grad-CAM algorithm, designed for exactly this purpose, yields some illuminating patterns. In particular, we use the grad-cam library, a Python implementation of the above algorithm designed for use with PyTorch convolutional neural networks, which produced the following heatmaps using the first three input layers:



Given the above, in addition to several other images tested, our model appears particularly sensitive to lighting and edges, as expected. That is, from manual observation, it is our opinion that deep fake images tend to exhibit unusual or physically impossible lighting conditions, with poorly-defined edges, an intuition our model seems to have adopted.

Model Evaluation and Analysis

We used a pretrained EfficientNet model as our base line with a small classifier linear model translating the 1000 outputs from EfficientNet into two, confidence on real and confidence on fake. This model was trained for over 10 hours running on a NVIDIA 4070 Laptop GPU with 8 GB of RAM. A total of 50 epochs were recorded and every 5 models would be saved. When evaluating the models to see the general trend, we observed the cross-entropy loss, which is the most accurate loss algorithm for efficient net models. Epoch 45 performed the best out of the saved, not making a single mistake on the 26000 photos for validation. Our 45th epoch model performed by far the best with not a single error out of the ones saved, but epoch 43 had the same loss ratings and epoch 44 had a perfect validation loss but not training. Likely because the EfficentNet model came partially trained the model trained to an incredibly fast

Interpretation of Results

In the world of AI image generation, it seems that the best tools for detection are other AI models. Image generation has issues with lighting and edges, so the model is easily able to catch those issues and flag the images as fake. For presentation or more testing, images can be taken and generated to show its performance. The perfect evaluation of the data set is impressive but may be a sign that it may be over tuned to the types of models used to

generate fake face images. That being said, its performance is a great sign that for use cases where detecting AI faces is important this model is incredible at fulfilling that job.

Conclusion and Future Work

The trained model was incredibly successful and accurate when evaluating real and generated images. The model trained can work as an effective back bone for AI generation image detection for a breath of applications. Despite the model's impressive results, the classifier, dataset, and training can all be improved in future testing. The classifier used worked well, but a different classifier may work better. Only a linear classifier was used, a sigmoid classifier may work better for the real versus generated section of the weight due to the exclusive properties of the classes. Due to the 0% error from the model, the model itself may be over trained on the types of models that were generating images. With more data and more different models generating head shots rather than just the three included could create a more robust model of real-world application. Lastly, with more data, training on AWS or google cloud servers as well as sharding the code could result in more robust and faster training. Lastly, for future real-world implementation, a GUI to drop photos in or a web extension that could analyze photos could prove to be an effective real-world application of this trained model.